

# Índice

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>Lo primero que llama la atención</b>               | <b>2</b>  |
| Gestión de Dependencias y Maven                       | 3         |
| <b>Ignorando archivos de compilación (.gitignore)</b> | <b>3</b>  |
| Crear un nuevo proyecto desde cero                    | 3         |
| <b>Sintaxis Básica</b>                                | <b>4</b>  |
| Llaves {} en vez de indentación                       | 4         |
| Punto y coma ; obligatorio                            | 5         |
| <b>Variables y Tipos</b>                              | <b>5</b>  |
| Java es fuertemente tipado                            | 5         |
| Immutable vs Mutable                                  | 6         |
| <b>Visibilidad (public / private)</b>                 | <b>6</b>  |
| Por qué existe este concepto                          | 6         |
| ¿Qué pasa si no ponemos modificador?                  | 7         |
| Niveles de visibilidad                                | 7         |
| <b>Clases y Herencia</b>                              | <b>7</b>  |
| Herencia con extends                                  | 7         |
| Clases abstractas                                     | 8         |
| Clases concretas (no abstractas)                      | 8         |
| <b>Métodos</b>                                        | <b>9</b>  |
| return es obligatorio                                 | 9         |
| void para métodos que no devuelven nada               | 9         |
| Parámetros de métodos                                 | 10        |
| <b>Colecciones</b>                                    | <b>10</b> |
| Java necesita tipo específico y new                   | 10        |
| Recorrer una colección                                | 10        |
| Otros tipos de colecciones                            | 10        |
| <b>Comparación Directa Wollok → Java</b>              | <b>11</b> |
| <b>Override</b>                                       | <b>12</b> |
| <b>Main - Punto de Entrada</b>                        | <b>12</b> |
| ¿Por qué tiene que ser exactamente ese método?        | 12        |
| ¿Cualquier clase puede tener el main?                 | 13        |
| ¿Por qué existe Main.java al principio?               | 13        |
| ¿Cuándo se puede eliminar?                            | 13        |
| Ejemplo de main que hace algo                         | 13        |
| ¿Qué pasa si el main está vacío?                      | 14        |
| ¿Qué pasa si no tenemos main?                         | 14        |
| <b>Constructores</b>                                  | <b>14</b> |
| Constructor en Java vs Wollok                         | 14        |
| ¿Por qué this?                                        | 15        |
| ¿Qué pasa si no escribimos constructor?               | 15        |
| Constructor en clase abstracta                        | 15        |
| ¿Por qué protected en vez de private?                 | 16        |

|                                                                |           |
|----------------------------------------------------------------|-----------|
| ¿Qué pasa si no escribimos constructor y lo necesitamos? ..... | 16        |
| <b>Reporte de Cobertura con JaCoCo y Servidor Local .....</b>  | <b>16</b> |
| ¿Qué es JaCoCo? .....                                          | 16        |
| El problema al abrir el reporte directamente .....             | 16        |
| La solución: Servidor local con Jetty .....                    | 17        |
| Cómo usarlo .....                                              | 17        |

- 1 Esta guía explica los conceptos básicos de Java comparándolos con WolloK, para
- 2 facilitar la traducción de un lenguaje a otro.

### 3 Lo primero que llama la atención

- 4 Lo primero que llama la atención es la estructura de directorios.

```

5
6 wtoj-plataforma-contenidos-CreadorSW-1
7   |
8   |— mvnw
9   |— mvnw.cmd
10  |— pom.xml
11  |— README.md
12  |— src
13     |— main
14        |— java
15           |— ar
16              |— edu
17                 |— unahur
18                    |— obj2
19                       |— w2j
20                          |— contenidos
21                             |— Contenido.java
22                             |— Peliculas.java
23                             |— series
24                                |— Episodio.java
25                                |— Serie.java
26                                |— Temporada.java
27                               |— Main.java
28  |— test
29     |— java

```

- 29 • Esa estructura es porque seguimos la convención de Maven.
- 30 • Cada clase en Java va en un archivo `.java` que se escribe con mayúscula.
- 31 • El `Main.java` viene con el repo como punto de entrada de ejemplo, pero no es
- 32 obligatorio para Maven. Esto se explica en detalle en la sección correspondiente.
- 33 • `mvnw` y `mvnw.cmd`: Scripts que permiten usar Maven sin instalarlo globalmente. El
- 34 primero es para Linux/Mac, el segundo para Windows.
- 35 • `pom.xml`: Archivo de configuración de Maven que define las dependencias y cómo
- 36 compilar el proyecto.

## Gestión de Dependencias y Maven

Java utiliza herramientas como **Maven** para gestionar las librerías externas. Los editores modernos (como Zed) detectan automáticamente la configuración del proyecto:

- **Detección:** El servidor de lenguaje (JDT.LS) busca el archivo `pom.xml`. Si existe, configura el proyecto siguiendo el estándar de Maven.
- **Maven Wrapper (mvnw):** Permite ejecutar Maven sin tenerlo instalado globalmente, asegurando que todos los desarrolladores usen la misma versión.
- **Repositorio Local:** Las dependencias se descargan una sola vez a la carpeta `~/.m2/repository`.
  - La primera compilación es lenta porque descarga todo de internet.
  - Las siguientes son rápidas porque Maven reutiliza lo que ya tiene en disco.
  - Esto permite trabajar offline una vez que las dependencias iniciales fueron obtenidas.

## Ignorando archivos de compilación (.gitignore)

Cuando subimos el proyecto a Git, **no** deben incluirse los archivos generados automáticamente:

- `target/`: Directorio donde Maven guarda los `.class` compilados y los resultados de los tests. Puede ocupar varios megabytes.
- `*.class`: Archivos de bytecode Java, por si compilamos un archivo suelto a mano fuera de Maven.

Estos archivos **no** deben subirse porque:

- Son generados automáticamente desde cualquier código fuente.
- Ocupan espacio innecesario.
- Cada desarrollador los genera localmente al compilar.

Todo lo demás **debe quedarse** en el repositorio:

- `.mvn/`, `mvnw`, `mvnw.cmd`, `pom.xml`, `src/`

Esto se configura en un archivo `.gitignore` en la raíz del proyecto.

## Crear un nuevo proyecto desde cero

Para no tener que crear la estructura de directorios a mano, Maven tiene un **archetype** que la genera automáticamente:

```
1 mvn archetype:generate \  
2   -DgroupId=ar.edu.unahur.obj2 \  
3   -DartifactId=nombre-del-proyecto \  
4   -DarchetypeArtifactId=maven-archetype-quickstart \  
5   -DinteractiveMode=false
```



73 Esto crea:

```
74 1 nombre-del-proyecto/  
75 2 |─ pom.xml  
76 3 |─ src/  
77 4   |─ main/java/ar/edu/unahur/obj2/App.java  
78 5   |─ test/java/ar/edu/unahur/obj2/AppTest.java
```

79 Después solo hay que ajustar el `pom.xml` a la versión de Java y las dependencias  
80 que se necesiten (JUnit 5, JaCoCo, etc.).

## 81 Sintaxis Básica

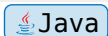
### 82 Llaves {} en vez de indentación

83 En Java, las llaves {} definen los bloques de código (clases, métodos, condiciones).

84 La indentación es solo visual, no sintáctica.

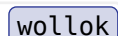
85 **Java:**

```
86 1 public class Contenido {  
87 2     private String titulo;  
88 3     private int costoBase;  
89 4  
90 5     public int costo() {  
91 6         return this.costoBase;  
92 7     }  
93 8 }
```



94 **Wollok:**

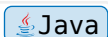
```
95 1 class Contenido {  
96 2     const titulo  
97 3     var costoBase  
98 4  
99 5     method costo() = costoBase  
100 6 }
```



101 ¿Qué pasa si no usamos las llaves?

102 **Java:**

```
103 1 public class Contenido {  
104 2     private String titulo // ERROR: missing } at end of class
```



105 El compilador tira error de sintaxis porque no puede determinar dónde termina el  
106 bloque.

## Punto y coma ; obligatorio

Java requiere ; al final de cada instrucción. Es obligatorio porque una línea puede contener varias instrucciones.

**Java:**

```
1 int x = 5;
2 x = x + 1;
```

 Java

**Wollok:**

```
1 var x = 5
2 x = x + 1
```

 wollok

¿Qué pasa si no ponemos ;?

**Java:**

```
1 int x = 5 // ERROR: syntax error, insert ';' to complete
  statement
2 x = x + 1;
```

 Java

El compilador informa exactamente dónde falta el ;.

## Variables y Tipos

### Java es fuertemente tipado

En Java **siempre** debemos indicar el tipo de la variable. No podemos dejar que el compilador lo infiera.

**Java:**

```
1 String titulo = "Recuerdos";
2 int costoBase = 20;
3 boolean esPopular = true;
4 double promedio = 22.5;
```

 Java

**Wollok:**

```
1 const titulo = "Recuerdos"
2 var costoBase = 20
3 var esPopular = true
4 var promedio = 22.5
```

 wollok

¿Qué pasa si usamos un tipo incorrecto?

**Java:**

```
138 1 String titulo = 123; // ERROR: incompatible types: int cannot
139   be converted to String
```

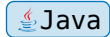


140 El compilador detecta el error antes de ejecutar el programa.

## 141 Immutable vs Mutable

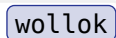
142 **Java:**

```
143 1 final String titulo = "Recuerdos"; // no se puede cambiar
144   (equivalente a const)
145 2 int costoBase = 20; // sí se puede cambiar
146   (equivalente a var)
```



147 **Wollok:**

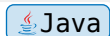
```
148 1 const titulo = "Recuerdos" // immutable
149 2 var costoBase = 20 // mutable
```



150 **¿Qué pasa si intentamos modificar una variable final?**

151 **Java:**

```
152 1 final String titulo = "Recuerdos";
153 2 titulo = "Otro título"; // ERROR: cannot assign a value to final
154   variable titulo
```



155 El compilador lanza error de compilación.

## 156 Visibilidad (public / private)

157 **Por qué existe este concepto**

158 El **encapsulamiento** protege los datos de modificaciones accidentales o  
159 incorrectas. Cada clase decide qué propiedades y métodos son accesibles desde  
160 afuera.

161 **Java:**

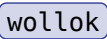
```
162 1 public class Contenido {
163 2     private String titulo; // solo la clase Contenido lo ve
164 3
165 4     public String getTitulo() { // acceso controlado
166 5         return this.titulo;
167 6     }
168 7
169 8     public void setTitulo(String nuevoTitulo) { // validación antes
170 9         de modificar
171         this.titulo = nuevoTitulo;
```



```
172 10    }
173 11 }
```

174 **Wollok:**

```
175 1 class Contenido {
176 2     const titulo
177 3     var costoBase
178 4 }
```



179 En Wollok no existe encapsulamiento: cualquiera puede ver y modificar cualquier  
180 atributo.

181 **¿Qué pasa si no ponemos modificador?**

182 **Java:**

```
183 1 class Contenido {
184 2     String titulo; // package-private (default)
185 3 }
```



186 Cualquier clase en el **mismo paquete** puede ver y modificar `titulo`. Es una  
187 vulnerabilidad porque se rompe el control sobre cómo se accede a los datos.

188 **Niveles de visibilidad**

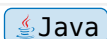
| 189 | Modificador    | Alcance                                         |
|-----|----------------|-------------------------------------------------|
| 190 | <b>public</b>  | Cualquier clase desde cualquier paquete         |
| 191 | <b>private</b> | Solo dentro de la misma clase                   |
| 192 | (ninguno)      | Package-private (solo clases del mismo paquete) |

193 **Clases y Herencia**

194 **Herencia con `extends`**

195 **Java:**

```
196 1 public class Pelicula extends Contenido {
197 2     // hereda titulo y costoBase automáticamente
198 3 }
```



199 **Wollok:**

```
200 1 class Pelicula inherits Contenido {
201 2     // vacía, hereda todo
202 3 }
```

wollok

203 **¿Qué pasa si heredamos de una clase que no existe?**

204 **Java:**

```
205 1 public class Pelicula extends Contenido { }
206 2 // ERROR: Contenido cannot be resolved to a type
```

Java

207 El compilador no encuentra la clase padre.

## 208 Clases abstractas

209 Una clase abstracta no se puede instanciar directamente. Sirve como base para  
210 otras clases.

211 **Java:**

```
212 1 public abstract class Contenido {
213 2     public abstract int costo(); // las subclasses DEBEN implementar
214 3 }
```

Java

215 **Wollok:**

```
216 1 class Contenido {
217 2     method costo() // sin cuerpo = abstracto
218 3 }
```

wollok

219 **¿Qué pasa si no implementamos el método abstracto?**

220 **Java:**

```
221 1 public class Pelicula extends Contenido {
222 2     // ERROR: The type Pelicula must implement the inherited abstract
223 3     method Contenido.costo()
224 }
```

Java

225 El compilador obliga a implementar todos los métodos abstractos.

## 226 Clases concretas (no abstractas)

227 **Java:**

```
228 1 public class Pelicula extends Contenido {
229 2     @Override
230 3     public int costo() {
231 4         return this.getCostoBase();
232 5     }
```

Java



233    6   }

234    **Wollok:**

```
235    1 class Pelicula inherits Contenido {  
236       2       // vacía, hereda el costo() de Contenido si existiera  
237       3   }
```

wollok

## 238    Métodos

### 239    **return es obligatorio**

240    Si declaramos un método que devuelve algo, **debemos** retornar un valor.

241    **Java:**

```
242    1 public int costo() {  
243       2       return this.costoBase;  
244       3   }
```

Java

245    **Wollok:**

```
246    1 method costo() = costoBase
```

wollok

247    **¿Qué pasa si no ponemos return?**

248    **Java:**

```
249    1 public int costo() {  
250       2       costoBase; // ERROR: missing return statement  
251       3   }
```

Java

252    El compilador detecta que el método promete devolver int pero no lo hace.

### 253    **void para métodos que no devuelven nada**

254    **Java:**

```
255    1 public void agregarTemporada(Temporada t) {  
256       2       temporadas.add(t);  
257       3   }
```

Java

258    **Wollok:**

```
259    1 method agregarTemporada(unaTemporada) {  
260       2       temporadas.add(unaTemporada)  
261       3   }
```

wollok

262    **¿Qué pasa si intentamos usar return en un método void?**

263 **Java:**

```
264 1 public void agregarTemporada(Temporada t) {  
265 2     return true; // ERROR: void methods cannot return a value  
266 3 }
```

 Java

267 **Parámetros de métodos**

268 **Java:**

```
269 1 public void agregarTemporada(Temporada unaTemporada) {  
270 2     temporadas.add(unaTemporada);  
271 3 }
```

 Java

272 **Wollok:**

```
273 1 method agregarTemporada(unaTemporada) {  
274 2     temporadas.add(unaTemporada)  
275 3 }
```

 wollok

276 **Colecciones**

277 **Java necesita tipo específico y new**

278 **Java:**

```
279 1 List<Temporada> temporadas = new ArrayList<>();  
280 2 temporadas.add(unaTemporada);  
281 3 temporadas.size();
```

 Java

282 **Wollok:**

```
283 1 const temporadas = []  
284 2 temporadas.add(unaTemporada)  
285 3 temporadas.size()
```

 wollok

286 **Recorrer una colección**

287 **Java:**

```
288 1 int suma = temporadas.stream().mapToInt(t -> t.costo()).sum();
```

 Java

289 **Wollok:**

```
290 1 var suma = temporadas.sum({ t => t.costo() })
```

 wollok

291 **Otros tipos de colecciones**

292 **Java:**

```

293 1 Set<String> titulos = new HashSet<>();
294 2 Map<String, Integer> duracion = new HashMap<>();

```

Java

295 **Wollok:**

```

296 1 // Wollok no distingue entre Set, Map, List
297 2 // usa simplemente listas
298 3 const titulos = []
299 4 const duracion = []

```

wollok

## 300 Comparación Directa Wollok → Java

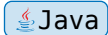
| 301        | Concepto         | Wollok                 | Java                                     |
|------------|------------------|------------------------|------------------------------------------|
| 302        | Inmutable        | const                  | final                                    |
| 303        | Mutable          | var                    | tipo (sin final)                         |
| 304        | Herencia         | class X inherits Y     | class X extends Y                        |
| 305        | Clase abstracta  | método sin cuerpo      | abstract class X                         |
| 306        | Method abstracto | method costo()         | public abstract int costo()              |
| 307        | Visibilidad      | no existe              | public / private                         |
| 308        | Lista            | []                     | new ArrayList<>()                        |
| 309        | Tamaño           | .size()                | .size()                                  |
| 310<br>311 | Sumar elementos  | .sum({x => x.costo()}) | .stream().mapToInt(x -> x.costo()).sum() |
| 312        | Punto y coma     | opcional               | obligatorio                              |
| 313        | Llaves           | no se usan             | obligatorias                             |
| 314        | Constructor      | automático             | hay que escribirlo                       |

## Override

En Java es **obligatorio** usar la anotación `@Override` cuando sobrescribimos un método de la clase padre.

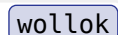
**Java:**

```
1 @Override
2 public int costo() {
3     return 3;
4 }
```



**Wollok:**

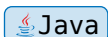
```
1 override method costo() {
2     return 3
3 }
```



¿Qué pasa si nos olvidamos de `@Override`?

**Java:**

```
1 public int costo() { // se supone que es override pero sin
2     return 3;
3 }
```



Si no es realmente un override (por ejemplo, si el método padre tiene otra firma), Java **no nos avisa**. Podemos tener un método completamente nuevo sin querer y el compilador no dice nada.

## Main - Punto de Entrada

Todo programa Java necesita un método `main` como punto de entrada. La JVM busca específicamente `public static void main(String[] args)` para saber dónde empezar.

¿Por qué tiene que ser exactamente ese método?

La firma debe ser literalmente `public static void main(String[] args)` porque es el contrato que la JVM espera:

- **public:** la JVM necesita poder acceder al método desde fuera de la clase.
- **static:** se ejecuta sin necesidad de crear una instancia de la clase.
- **void:** no devuelve nada a la JVM.
- **main:** es el nombre exacto que la JVM busca.
- **String[] args:** recibe los argumentos que se pasan por línea de comandos.

Si cambiamos algo (por ejemplo, `public static void start(String[] args)`), la JVM no lo reconoce y tira error.

## ¿Cualquier clase puede tener el main?

Sí, pero no literalmente cualquier clase: tiene que ser una clase que contenga el método exacto `public static void main(String[] args)`. El nombre de la clase puede ser cualquiera (`Main.java`, `App.java`, `Launcher.java`, etc.). Maven y Java no obligan a que se llame `Main`, ese nombre es solo una convención.

Poner el `main` en una clase de dominio (como `Contenido.java` o `Plataforma.java`) funciona, pero no es una buena práctica porque mezcla la infraestructura (punto de entrada) con la lógica de negocio. Es mejor mantenerlo en una clase separada.

## ¿Por qué existe `Main.java` al principio?

El `Main.java` que viene en el repositorio inicial es una decisión de los docentes, no un requisito de Maven. Maven compila el proyecto y genera la estructura de directorios basándose únicamente en el archivo `pom.xml` y en la convención de directorios (`src/main/java`, `src/test/java`, etc.). No necesita un archivo `Main.java` para funcionar.

## ¿Cuándo se puede eliminar?

Si nuestro proyecto solo contiene clases de dominio y tests con JUnit, podemos eliminar el `Main.java` sin problemas. La ejecución de los tests se realiza mediante `mvn test` a través del plugin Surefire, que no requiere un método `main`.

Es importante entender que al eliminar el `Main.java`, nuestro proyecto como programa ejecutable no está haciendo nada. Los tests verifican que las clases de dominio funcionen correctamente, pero no ejecutan el sistema: instancian objetos, les mandan mensajes y comprueban resultados, pero no ponen en marcha una aplicación. Si queremos que nuestro proyecto haga algo como programa (por ejemplo, mostrar información en consola o iniciar una interfaz), necesitamos un `main` que cree objetos y coordine el comportamiento. Solo necesitamos un `main` si vamos a ejecutar nuestro programa como una aplicación standalone (por ejemplo, con `java -jar` o `mvn exec:java`).

## Ejemplo de `main` que hace algo

El `main` nunca debe tener toda la lógica. Solo crea objetos y delega.

```
1 package ar.edu.unahur.obj2.w2j;
2
311 3 import ar.edu.unahur.obj2.w2j.contenidos.Pelicula;
312 4 import ar.edu.unahur.obj2.w2j.planes.PlanBasico;
313 5 import ar.edu.unahur.obj2.w2j.planes.Usuario;
314 6
315 7 public class Main {
316 8     public static void main(String[] args) {
317 9         // Crear contenidos
```

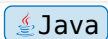


```
388 10      Pelicula pelicula1 = new Pelicula("Recuerdos", 20.0);
389 11      Pelicula pelicula2 = new Pelicula("Avatar", 30.0);
390 12
391 13      // Crear un usuario con plan básico
392 14      PlanBasico plan = new PlanBasico(100.0);
393 15      Usuario usuario = new Usuario("Juan", plan);
394 16
395 17      // El usuario mira contenidos
396 18      usuario.verContenido(pelicula1);
397 19      usuario.verContenido(pelicula2);
398 20
399 21      // Mostrar resultados
400 22      System.out.println("Usuario: " + usuario.getNombre());
401 23      System.out.println("Costo total del plan: " +
402 24      usuario.getPlan().getCosto());
403 25  }
```

## 405 ¿Qué pasa si el main está vacío?

406 El programa compila y corre, pero no hace nada visible. Las clases de dominio  
407 existen en el código, pero si nadie las instancia dentro del main, simplemente  
408 «están ahí».

```
409 1 public static void main(String[] args) {
410 2     // vacío
411 3 }
```



## 412 ¿Qué pasa si no tenemos main?

413 Si intentamos ejecutar el programa con java o mvn exec:java pero ninguna clase  
414 tiene el método main, obtenemos:

```
415 1 Error: Main method not found in class Contenido, please define the main
416 2 method as:
417 3 public static void main(String[] args)
```

## 418 Constructores

419 En Java, si necesitamos inicializar objetos con parámetros, debemos escribir el  
420 constructor explícitamente. Wollok lo hace automáticamente, Java no.

## 421 Constructor en Java vs Wollok

422 **Java:**

```
423 1 public class Contenido {
424 2     private final String titulo;
425 3     private double costoBase;
426 4
427 5     public Contenido(String titulo, double costoBase) {
428 6         this.titulo = titulo;
429 7         this.costoBase = costoBase;
430 8     }
431 9 }
```

 Java

432 **Wollok:**

```
433 1 class Contenido {
434 2     const titulo
435 3     var costoBase
436 4 }
437 5 // Wollok genera automáticamente el constructor con los parámetros
```

 wollok

438 **¿Por qué this?**

439 this.titulo significa «el campo titulo de este objeto». Sin this, Java no sabe si  
440 titulo se refiere al parámetro o al campo.

```
441 1 public Contenido(String titulo, double costoBase) {
442 2     this.titulo = titulo;           // this.titulo = campo, titulo =
443 3     parámetro
444 4     this.costoBase = costoBase;
445 5 }
```

 Java

446 **¿Qué pasa si no escribimos constructor?**

```
447 1 public class Contenido {
448 2     private String titulo;
449 3     private double costoBase;
450 4 }
451 5 // Java crea automáticamente: Contenido() {}
```

 Java

452 Si no definimos ningún constructor, Java crea uno automático sin parámetros. Pero  
453 si definimos **al menos uno con parámetros**, el automático desaparece.

454 **Constructor en clase abstracta**

```
455 1 public abstract class Contenido {
456 2     private final String titulo;
457 3     protected double costoBase; // protected para que las subclases
458 4     accedan
```

 Java

```

459 4
460 5     public Contenido(String titulo, double costoBase) {
461 6         this.titulo = titulo;
462 7         this.costoBase = costoBase;
463 8     }
464 9 }

```

465 **¿Por qué `protected` en vez de `private`?**

| Modificador            | Quién puede verlo        |
|------------------------|--------------------------|
| <code>private</code>   | Solo la clase misma      |
| <code>protected</code> | La clase y sus subclases |

469 Si `costoBase` fuera `private`, `Pelicula` (que hereda de `Contenido`) no podría acceder  
470 a `costoBase`.

471 **Wollok:** No existe este concepto. Todos los atributos son accesibles.

472 **¿Qué pasa si no escribimos constructor y lo necesitamos?**

```

473 1  Contenido c = new Contenido(); // ERROR: no default constructor
474 exists

```

475 Si definimos un constructor con parámetros y necesitamos el sin parámetros,  
476 debemos escribirlo nosotros.

## 477 Reporte de Cobertura con JaCoCo y Servidor 478 Local

479 **¿Qué es JaCoCo?**

480 JaCoCo es una herramienta que genera reportes de cobertura de código, es decir,  
481 indica qué partes de nuestro código fueron ejecutadas por los tests y cuáles no. El  
482 reporte se genera como una página HTML dentro de la carpeta `target/site/`  
483 `jacoco/`.

484 **El problema al abrir el reporte directamente**

485 Al abrir el archivo `index.html` del reporte haciendo doble clic, el navegador lo  
486 carga con el protocolo `file:///`. Algunos navegadores (como Chromium o  
487 Ungogled Chromium) bloquean la carga de recursos locales (CSS y JavaScript)  
488 por políticas de seguridad. Esto hace que el reporte se vea incompleto o sin estilos.  
489 Firefox, en cambio, suele permitirlo.



## La solución: Servidor local con Jetty

Para evitar este problema, podemos levantar un servidor web local que sirva el reporte mediante el protocolo `http://`. Esto se logra agregando el plugin de Jetty al archivo `pom.xml`:

```
1  <plugin>
2      <groupId>org.eclipse.jetty</groupId>
3      <artifactId>jetty-maven-plugin</artifactId>
4      <version>11.0.16</version>
5      <configuration>
6          <supportedPackagings>
7              <supportedPackaging>jar</supportedPackaging>
8          </supportedPackagings>
9          <webApp>
10             <resourceBase>${project.build.directory}/site/jacoco</
11             resourceBase>
12          </webApp>
13          <httpConnector>
14              <port>8080</port>
15          </httpConnector>
16      </configuration>
17  </plugin>
```

## Cómo usarlo

Una vez que el reporte de JaCoCo fue generado (con `mvn test`), ejecutamos:

```
1  mvn jetty:run
```

Esto levanta un servidor local en `http://localhost:8080`. Al abrir esa dirección en cualquier navegador, el reporte se visualiza correctamente con todos sus estilos y funcionalidades.

La primera vez que se ejecuta, Maven descarga el plugin de Jetty y sus dependencias. Esto sucede una sola vez y se almacena en el repositorio local (`~/.m2/repository`). Las siguientes ejecuciones son instantáneas, incluso sin conexión a internet.